Musterlösung zur Klausur zu OP_SA2

1. Aufgabe: Versionierung / Versionsverwaltung (5 Punkte)

Erläutern Sie kurz (2-3 Sätze): Was versteht man im Zusammenhang mit Versionsverwaltung unter *Optimistic Locking / Pessimistic Locking?*

Basisanwendung

Für die folgenden Aufgaben ist eine Anwendung zur Verwaltung von Teppichen als Basis gegeben, siehe die folgenden Seiten.

Klasse Teppichverwaltung

```
public class Teppichverwaltung {

  private Teppich[] teppiche = new Teppich[2];

  private void fuelleTeppiche() {
        this.teppiche[0] = new Teppich("111_high", true, 180, 300, 590);
        this.teppiche[1] = new Teppich("222_low", false, 150, 270, 490);
  }

  private void schreibeTeppicheInCsvDatei() {
        try {
              Creator creator = new CsvExportCreator();
              Export export = creator.factoryMethod();
              export.schreibeTeppicheInDatei(this.teppiche);
        }
        catch(Exception exc) {
              exc.printStackTrace();
        }
  }

  public static void main(String[] args) {
        Teppichverwaltung teppichverwaltung = new Teppichverwaltung();
        teppichverwaltung.fuelleTeppiche();
        teppichverwaltung.schreibeTeppicheInCsvDatei();
  }
}
```

Klasse Teppich

```
public class Teppich {

  private String artikelnummer;
  private boolean hochflorTeppich;
  private int breite;
  private int laenge;
  private int preis;
```

```
  public String gibAttributeAus(String trenner) {
        String ergebnis = this.getArtikelnummer() + trenner
              + this.isHochflorTeppich() + trenner
              + this.getBreite() + trenner
              + this.getLaenge() + trenner
              + this.getPreis();
        return ergebnis;
  }
  // Konstruktor mit Parametern und get- und set-Methoden
  …
}
```

Klasse DatenCsvExport

```
import java.io.*;
public class DatenCsvExport extends Export{

  @Override
  public void schreibeTeppicheInDatei(Teppich[] teppiche)
        throws IOException{
        BufferedWriter aus
              = new BufferedWriter(new FileWriter("Teppiche.csv"));
        for(int i = 0; i < teppiche.length; i++) {
              aus.write(teppiche[i].gibAttributeAus(";") + "\n");
        }
        aus.close();
  }
}
```

Klasse Export

```
import java.io.IOException;
public abstract class Export {

    public abstract void schreibeTeppicheInDatei(Teppich[] teppiche)
         throws IOException;
   }
}
```

Klasse Creator

```
public abstract class Creator {
    public abstract Export factoryMethod();
}
```

Klasse CsvExportCreator

```
public class CsvExportCreator extends Creator {
  @Override
  public Export factoryMethod() {
        return new DatenCsvExport();
  }
}
```

2. Aufgabe: Design Patterns (40 Punkte)

Grundlage ist die Basisanwendung zur Verwaltung von Teppichen

a) Geben Sie das UML Klassendiagramm an, welches sich aus den Klassen der Basisanwendung zur Verwaltung von Teppichen ergibt. Sie brauchen packages, get- und set-Methoden, Konstruktoren und Exceptions nicht zu berücksichtigen.

b) Nennen und erläutern Sie dasjenige Design Pattern, welches in diesem Beispiel umgesetzt wird.
   i) Welche Klassen realisieren das Design Pattern, welche benutzen es?
   ii) Was ist der Nutzen des Design Pattern bezogen auf dieses Beispiel?
   iii) Nennen Sie ein Beispiel für eine Weiterentwicklung der Anwendung, welche durch dieses Design Pattern unterstützt wird. Geben Sie den Quellcode dieser Weiterentwicklung an (, Erweiterung des Design Pattern und auch die Benutzung der Erweiterung).

Lösung

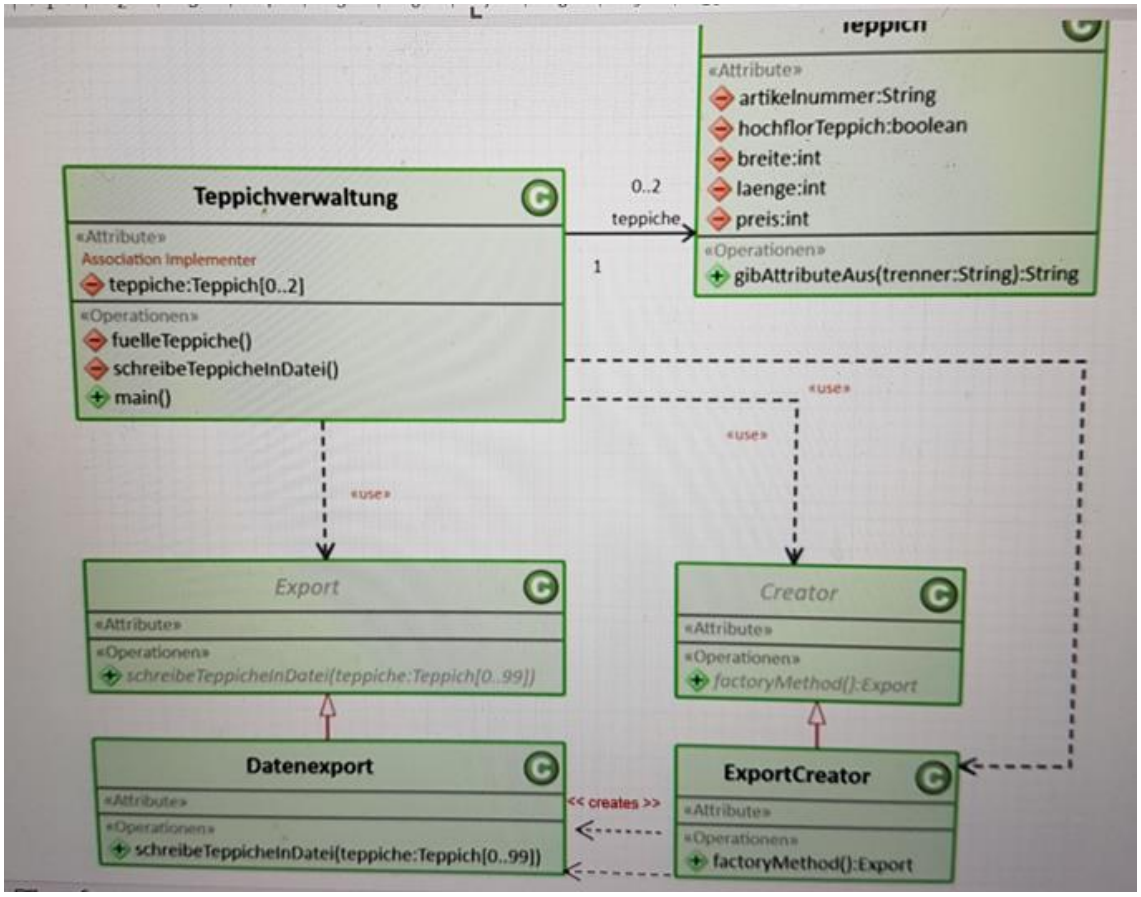
Zu a) Fabrikmethode-Pattern **6 Punkte**

Klassen, die das Pattern umsetzen: Creator, CsvExportCreator, DatenCsvExport, Export

Klasse, die das Pattern nutzt: Teppichvewaltung

Zu b) i) **4 Punkte**

Zu b) ii) **12 Punkte**



Achtung: Klassennamen geringfügig anders!

Elemente	**Punkte**
4 normale Klassen inklusive Attribute und Methoden	4
2 abstrakte Klassen inklusive Methoden	2
1 Assoziation	1
2 Generalisierungen	2
4 Verwendungen (2-3 x use und 1 x crfeate: 2 + 1)	3
insgesamt	**12**

Zu b) iii) **18 Punkte**

Klasse TxtExportCreator

```
public class TxtExportCreator extends Creator {                    1

   @Override                                                       0,5
   public Export factoryMethod() {                                 1
         return new DatenTxtExport();                              1
   }
}
```

Klasse DatenTxtExport

```
import java.io.*;                                                  0,5

public class DatenTxtExport extends Export{                        1

   @Override                                                       1
   public void schreibeTeppicheInDatei(Teppich[] teppiche)         1
         throws IOException{                                       0,5
         BufferedWriter aus
               = new BufferedWriter(new FileWriter("Teppiche.txt"));1
         aus.write("Artikelnr  Hochflor?  Breite  Laenge  Preis\n");
         for(int i = 0; i < teppiche.length; i++) {                1
               aus.write(teppiche[i].gibAttributeAus("  ") + "\n"); 1
         }
         aus.close();                                              0,5
   }
}
```

In Klasse Teppichverwaltung

```
   private void schreibeTeppicheInTxtDatei() {                     1
         try {                                                     0,5
               Creator creator = new TxtExportCreator();           1
               Export export = creator.factoryMethod();            1
               export.schreibeTeppicheInDatei(this.teppiche);      1
         }
         catch(Exception exc) {                                    0,5
               exc.printStackTrace();                              0,5
         }
   }

   public static void main(String[] args) {                        0,5
         Teppichverwaltung teppichverwaltung = new Teppichverwaltung();
         teppichverwaltung.fuelleTeppiche();
         teppichverwaltung.schreibeTeppicheInCsvDatei();
         teppichverwaltung.schreibeTeppicheInTxtDatei();           1
   }                                                               18
```

3. Aufgabe: Vertiefung Java (30 Punkte)

Gegeben ist die Basisanwendung zur Verwaltung von Teppichen und eine weitere Klasse *OutdoorTeppich*. Packages und Importe werden hier nicht berücksichtigt.

```
public class OutdoorTeppich extends Teppich{

  private boolean wasserabweisend;

  @Override
  public String gibAttributeAus(String trenner) {
        String ergebnis = super.gibAttributeAus(trenner) + trenner
              + this.isWasserabweisend();
        return ergebnis;
  }

  // Konstruktor mit Parametern und get- und set-Methode
  …
}
```

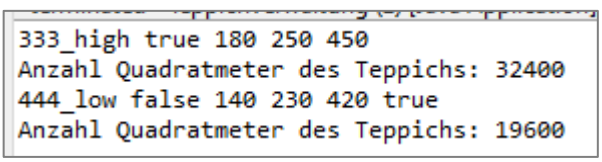
a) Generics, for each

Geben Sie den Quellcode an, um in der Klasse *Teppichverwaltung* das Attribut *teppiche* vom Typ *Teppich[ ]* zu ersetzen durch ein Objekt vom Typ *ArrayList* unter Verwendung von Generics. Geben Sie weiterhin den neuen Quellcode der Methode *fuelleTeppiche* der Klasse *Teppichverwaltung* und der Methode *schreibeTeppicheInDatei* der Klasse *DatenCsvExport* unter Verwendung einer for each Schleife an. (Mit *for each* ist nicht die Methode *forEach* gemeint!) In welcher Klasse ist weiterhin eine Änderung notwendig? (Nennen Sie nur den Klassennamen.) Importe brauchen Sie nicht zu berücksichtigen.

b) Generics mit Typeinschränkungen

Geben Sie den Quellcode einer Klasse *TeppichUtil* an, welche ein Generic *T* enthält, für welches nur ein Objekt vom Typ *Teppich* oder einer Ableitung von *Teppich* eingesetzt werden kann. Es gibt ein privates Attribut *teppichBasis* vom Typ *T*, einen Konstruktor, welcher das Attribut mit einem Wert belegt, und eine Methode *gibAttributeUndQuadratmeterTeppichBasisInKonsoleAus* ohne Parameter und Rückgabewert. Diese schreibt die Attribute von *T* und auch die Quadratmeter in die Konsole, siehe unten.

Ergänzen Sie die *main*-Methode der Klasse *Teppichverwaltung* folgendermaßen. Erstellen Sie ein Objekt *teppich* vom Typ *Teppich* und ein Objekt *outdoorTeppich* vom Typ OutdoorTeppich und zwei Objekte vom Typ *TeppichUtil* unter Verwendung der Objekte *teppich* und *outdoorTeppich* und erzeugen Sie die folgende Ausgabe.



Lösung

Zu a)

In Klasse Teppichverwaltung

```
private ArrayList<Teppich> teppiche = new ArrayList<>();                2

private void fuelleTeppiche() {
      this.teppiche.add(new Teppich("111_high", true, 180, 300, 590)); 1
      this.teppiche.add(new Teppich("222_low", false, 150, 270, 490)); 1
}
```

In Klasse DatenCsvExport

```
@Override
public void schreibeTeppicheInDatei(ArrayList<Teppich> teppiche)        2
      throws IOException{
      BufferedWriter aus
            = new BufferedWriter(new FileWriter("Teppiche.csv"));
      for(Teppich teppich : teppiche) {                                 2,5
            aus.write(teppich.gibAttributeAus(";") + "\n");             1,5
      }
      aus.close();
}

}
```

Eine Änderung in der Klasse Export ist notwendig: 1

*schreibeTeppicheInDatei(**ArrayList<Teppich>** teppiche))* 11

Zu b)

Klasse TeppichUtil

```
public class TeppichUtil<T extends Teppich> {                          3

    private T teppichBasis;                                            1,5

    public TeppichUtil(T teppichBasis) {                               2
          super();
          this.teppichBasis = teppichBasis;                            1
    }

    public void gibAttributeUndQuadratmeterTeppichInKonsoleAus() {     1
          System.out.println(teppichBasis.gibAttributeAus(" ")
                + "\nAnzahl Quadratmeter des Teppichs: "
          + (teppichBasis.getBreite() * teppichBasis.getBreite()));    2,5
    }
}
```

In der main-Methode der Klasse Teppichverwaltung

```
Teppich teppich = new Teppich("333_high", true, 180, 250, 450);   1
OutdoorTeppich outdoorTeppich
    = new OutdoorTeppich("444_low", false, 140, 230, 420, true);  1
TeppichUtil<Teppich> teppichUtil0
    = new TeppichUtil<Teppich>(teppich);                          2
TeppichUtil<OutdoorTeppich> teppichUtil1
    = new TeppichUtil<OutdoorTeppich>(outdoorTeppich);            2
teppichUtil0.gibAttributeUndQuadratmeterTeppichInKonsoleAus();    1
teppichUtil1.gibAttributeUndQuadratmeterTeppichInKonsoleAus();    1
                                                                  19
```

4. Aufgabe: JUnit 5 und Zustandsbasierter Test (25 Punkte)

Gegeben ist die Basisanwendung zur Verwaltung von Teppichen.

JUnit 5 15 Punkte

Ergänzen Sie den vorliegenden Quellcode eines JUnit Tests. Der Konstruktor der Klasse *Teppich* soll durch diese JUnit Testklasse daraufhin getestet werden, ob die Artikelnummer beim Kreieren eines Teppichs richtig gesetzt wird. Weiterhin soll der Konstruktor daraufhin getestet werden, ob bei der Eingabe eines negativen Preises eine *IllegalArgumentException* mit der Meldung *Der Preis darf nicht negativ sein.* geworfen wird. Benutzen Sie aus der Klasse *Assertions*
assertTrue(BooleanSupplier conditionSupplier, Supplier<String> messageSupplier),
assertEquals(String actual, String expected und
<T> assertThrows(Class<T> clazz, Executable executable).

```
import static org.junit.jupiter.api.Assertions.*;
import  org.junit.jupiter.api.*;
public class TeppichTest{

   // Ergaenzen!!!
   private Teppich teppich;                                          1,5

   @AfterEach
   public void tearDown() throws Exception{
         // Ergaenzen!!!
         this.teppich = null;                                         1

   }

   @Test
   public void test() throws Exception{
         // Ergaenzen!!!
         this.teppich = new Teppich("222_low", false, 150, 270, 490);   1,5

         assertTrue(()-> "222_low".equals(teppich.getArtikelnummer()),
               ()-> "Die Artikelnummer wird nicht richtig belegt.");      4

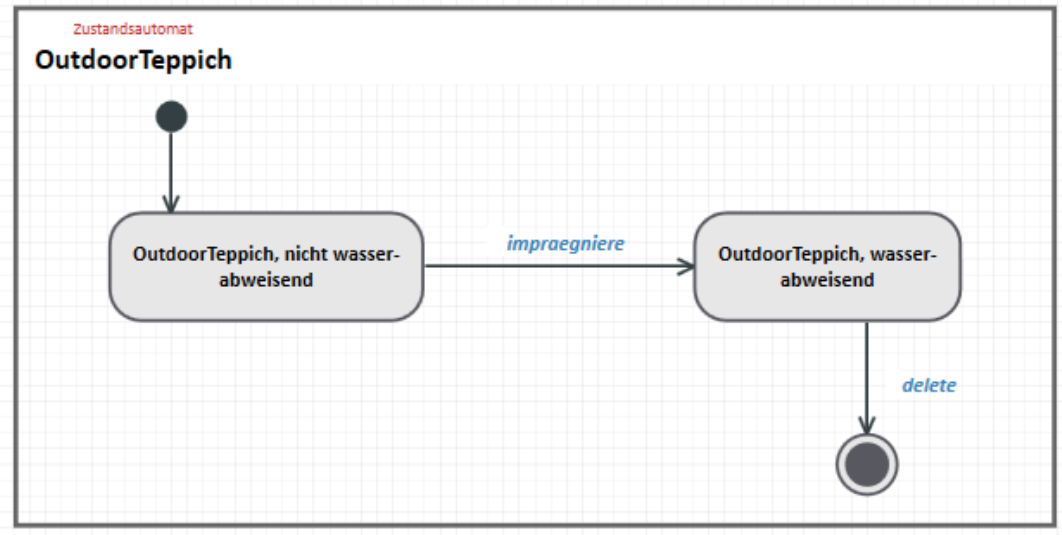
   }

   @Test
   public void testException() throws Exception{
         // Ergaenzen!!!
         Throwable exc = assertThrows(IllegalArgumentException.class,
               () -> {
               new Teppich("111_high", true, 180, 300, -1);
         });                                                          4

         Assertions.assertEquals("Der Preis darf nicht negativ sein.",
               exc.getMessage());                                     3
   }                                                                  15
}
```

Zustandsbasierter Test **10 Punkte**

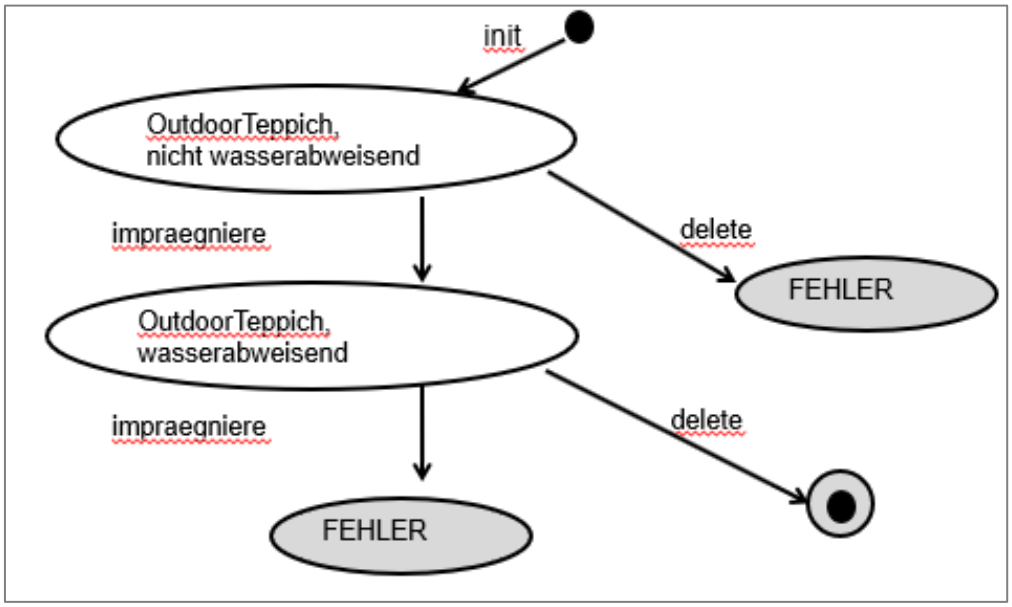
Gegeben ist ein UML Zustandsautomat, welcher die Zustände eines Objekts vom Typ *OutdoorTeppich* der Aufgabe 3 modelliert. Geben Sie den Übergangsbaum an. Geben Sie zwei Testfälle inklusive Sollergebnis an, die sich aus dem Übergangsbaum ergeben.



Lösung

Übergangsbaum: **6 Punkte**

2 Testfälle: **je 2 Punkte**



Testfälle:

init / impraegniere / delete -> Ein OutdoorTeppich-Objekt, nicht wasserabweisend, wurde angelegt, imprägniert und dann gelöscht.

init / delete -> Ein OutdoorTeppich-Objekt, nicht wasserabweisend, wurde angelegt und dann gelöscht. Fehlermeldung beim Löschversuch.

5. Aufgabe: SOLID und Clean Code (20 Punkte)

Gegeben ist die Anwendung zur Verwaltung von Teppichen inklusive der Klasse *OutdoorTeppich* aus der 3. Aufgabe.

a) 6 Punkte

Erläutern Sie das *Liskov Substitution Principle.* Nennen Sie ein Beispiel unter Verwendung der Klassen *Teppich* und *OutdoorTeppich*.

Unterklassen sollen anstelle ihrer Oberklassen einsetzbar sein. Eine Instanz einer Unterklasse muss sich so verhalten, dass jemand, der meint, ein Objekt der Oberklasse vor sich zu haben, nicht durch unerwartetes Verhalten überrascht wird, wenn es sich dabei tatsächlich um ein Objekt der Unterklasse handelt.

Beispiel: Die Methode gibAttributeAus darf sich im Wesentlichen nicht unterunterschiedlich verhalten bezüglich *Teppich* und *OutdoorTeppich*.

b) 4 Punkte

Wieso trägt das *Liskov Substitution Principle* zu einer höheren Wartbarkeit und Lebensdauer von Software und zu einer Reduktion der Fehleranfälligkeit bei (2-3 Sätze)?

Durch das Prinzip wird garantiert, dass Operationen vom Typ *Oberklasse*, die auf ein Objekt des Typs *Unterklasse* angewendet werden, auch korrekt ausgeführt werden.

c) 5 Punkte

Nennen und erläutern Sie kurz einen Wert aus Clean Code. Geben Sie an, ob, und wenn ja, wie in dem vorgegebenen Beispiel zur Verwaltung von Teppichen in den Aufgaben 2, 3 oder 4 eine Maßnahme zur Erfüllung dieses Werts umgesetzt wurde.

Beispiel: *Korrektheit:* Der Code soll frei von Fehlern sein, umgesetzte Maßnahme: JUnit Test und Zustandsbasierter Softwaretest.

d) 5 Punkte

Erläutern Sie kurz das Prinzip *Integration Operation Segregation Principle (IOSP)*, welches dem roten Grad aus Clean Code zugeordnet ist. Was ist mit *Integration* gemeint, was mit *Operation*? Ist das Prinzip bereits in dem Beispiel zu *Teppichen* umgesetzt worden? Begründen Sie kurz Ihre Antwort.

Integration Operation Segregation Principle (IOSP) bedeutet, dass man Integration (Verwaltung, Koordination) von Operation (Berechungen, Programmlogik) trennen sollte. Das Prinzip wurde in dem Beispiel umgesetzt, da es bisher keine nennenswerten Berechungen gibt.